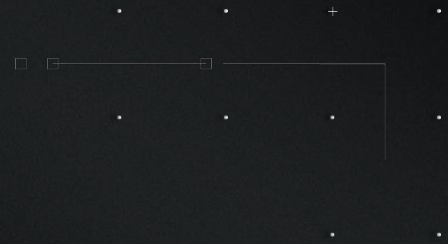


Programação Orientada a Objetos

C#

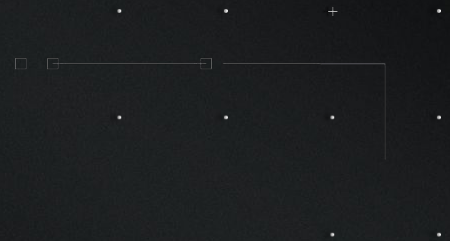


Agenda do dia

1. Modificadores de Acesso

2. Herança

3. Interfaces



Modificadores de Acesso

- Os modificadores de acesso determinam a visibilidade de classes, membros de classes, métodos, propriedades e outros elementos em C#.

Existem quatro tipos principais:

public: A classe ou membro é acessível de qualquer lugar.

private: A classe ou membro só é acessível dentro da própria classe.

protected: O membro é acessível dentro da própria classe e suas classes derivadas.

internal: O membro é acessível apenas dentro do mesmo assembly (compilação).

Exemplo

https://github.com/profalexresende/CSharp_modificadoresAcesso.git

Classe ContaBancaria.cs

```
internal class ContaBancaria
```

```
{  
    // Exemplo de membros com diferentes modificadores de acesso  
    2 references  
    public int NumeroConta { get; set; }  
    7 references  
    private double Saldo { get; set; }  
    1 reference  
    protected string Titular { get; set; }  
    1 reference  
    internal bool Ativa { get; set; }  
  
    1 reference  
    public ContaBancaria(int numeroConta, double saldoInicial, string titular)  
    {  
        NumeroConta = numeroConta;  
        Saldo = saldoInicial;  
        Titular = titular;  
        Ativa = true;  
    }  
  
    1 reference  
    public void Depositar(double valor)  
    {  
        Saldo += valor;  
        Console.WriteLine($"Depósito de {valor} realizado. Novo saldo: {Saldo}");  
    }  
}
```

```
private void Sacar(double valor)
```

```
{  
    if (valor <= Saldo)  
    {  
        Saldo -= valor;  
        Console.WriteLine($"Saque de {valor} realizado. Novo saldo: {Saldo}");  
    }  
    else  
    {  
        Console.WriteLine("Saldo insuficiente para saque.");  
    }  
}
```

```
0 references
```

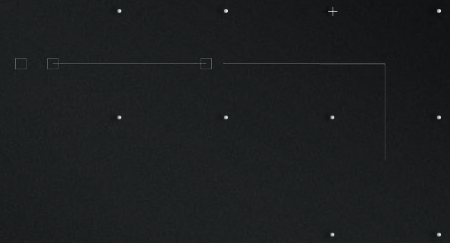
```
protected void ExibirSaldo()
```

```
{  
    Console.WriteLine($"Saldo da conta {NumeroConta}: {Saldo}");  
}
```

Classe Program.cs

```
2 using modificadoresAcesso;
3
4 ContaBancaria minhaConta = new ContaBancaria(1234, 1000, "Alex");
5
6 bool continuar = true;
7
8 while (continuar)
9 {
10     Console.WriteLine("\nMenu:");
11     Console.WriteLine("1. Exibir Saldo");
12     Console.WriteLine("2. Depositar");
13     Console.WriteLine("3. Sacar");
14     Console.WriteLine("4. Sair");
15
16     Console.Write("Escolha uma opção: ");
17     string opcao = Console.ReadLine();
18
19     switch (opcao)
20     {
21     case "1":
22         //minhaConta.ExibirSaldo(); ERRO: exibirSaldo não é acessível
23         break;
24
25     case "2":
26         Console.Write("Digite o valor para depósito: ");
27         double valorDeposito = Convert.ToDouble(Console.ReadLine());
28         minhaConta.Depositar(valorDeposito);
29         break;
30
31     case "3":
32         Console.Write("Digite o valor para saque: ");
33         double valorSaque = Convert.ToDouble(Console.ReadLine());
34         //minhaConta.Sacar(valorSaque); ERRO o sacar não é acessível
35         break;
36
37     case "4":
38         continuar = false;
39         break;
40
41     default:
42         Console.WriteLine("Opção inválida. Tente novamente.");
43         break;
44     }
45 }
```


Herança



- A herança é um conceito fundamental na programação orientada a objetos que permite que uma classe herde características e comportamentos de outra classe. A classe que é herdada é chamada de **classe base** ou **superclasse**, e a classe que herda é chamada de **classe derivada** ou **subclasse**.

Exemplo

https://github.com/profalexresende/CSharp_heranca.git

Classe Base Veiculo.cs

```
9      internal class Veiculo
10     {
11         //SUPERCLASSE
12         1 reference
13         public string Marca { get; set; }
14         1 reference
15         public string Modelo { get; set; }
16
17         0 references
18         public void Dirigir()
19         {
20             Console.WriteLine($"Dirigindo um {Marca} {Modelo}");
21         }
22     }
```


Classe Derivada Carro.cs

```
9      internal class Carro: Veiculo
10     {
11         //Classe Derivada
12         public int Portas { get; set; }
13         public void AbrirPortas()
14         {
15             Console.WriteLine($"Abrindo as {Portas} portas do carro");
16         }
17     }
```

Classe Derivada Moto.cs

```
9      internal class Moto: Veiculo
10     {
11         //Classe Derivada
12         public bool PartidaEletrica { get; set; }
13
14         public void AcionarPartidaEletrica()
15         {
16             if (PartidaEletrica)
17             {
18                 Console.WriteLine("Acionando a partida elétrica da moto");
19             }
20             else
21             {
22                 Console.WriteLine("A moto não possui partida elétrica");
23             }
24         }
25     }
```

Program.cs

```
1 // See https://aka.ms/new-console-template for more information
2 // Criando uma instância da classe derivada (Carro)
3 using heranca;
4
5 Carro meuCarro = new Carro
6 {
7     Marca = "Toyota",
8     Modelo = "Corolla",
9     Portas = 4
10 };
11
12 meuCarro.Dirigir(); // Método da classe base
13 meuCarro.AbrirPortas(); // Método da classe derivada
14
15 // Criando uma instância da outra classe derivada (Moto)
16 Moto minhaMoto = new Moto
17 {
18     Marca = "Honda",
19     Modelo = "CB500",
20     PartidaEletrica = true
21 };
22
23 minhaMoto.Dirigir(); // Método da classe base
24 minhaMoto.AcionarPartidaEletrica(); // Método da classe derivada
25
```

Interfaces



Interfaces definem contratos que as classes devem seguir, especificando métodos, propriedades e eventos que as classes devem implementar.

Exemplo

https://github.com/profalexresende/CSharp_interfaces.git



Interface IPessoa.cs

```
{ } 1  using System;
    2  using System.Collections.Generic;
    3  using System.Linq;
    4  using System.Text;
    5  using System.Threading.Tasks;
    6
    7  namespace interfaces
    8  {
    9      4 references
      internal interface IPessoa
     10      {
     11          6 references
          string Nome { get; set; }
     12          4 references
          void Apresentar();
     13      }
     14  }
```

Classe Funcionario.cs e Cliente.cs

```
9  internal class Funcionario: IPessoa
10 {
11
12     3 references
13     public string Nome { get; set; }
14
15     3 references
16     public void Apresentar()
17     {
18         Console.WriteLine($"Funcionário: {Nome}");
19     }
20 }
```

```
9  internal class Cliente: IPessoa
10 {
11
12     3 references
13     public string Nome { get; set; }
14
15     3 references
16     public void Apresentar()
17     {
18         Console.WriteLine($"Cliente: {Nome}");
19     }
20 }
```


OBRIGADO



profalex.deus@fiap.com.br



/alexsanderresende

FIAP

Copyright © 2018 | Professor (a) Alex Sander Resende de Deus

Todos os direitos reservados. A reprodução ou divulgação total ou parcial deste documento é expressamente proibida sem o consentimento formal, por escrito, do professor/autor.

